

Using uv in This Course

uv is the standard Python project tool for this course. You will usually use it to set up the course environment, run Python code, run notebooks, and run tests in a consistent way.

What uv is

uv is a Python package and project manager. In practice, that means it helps manage:

- project dependencies
- virtual environments
- package versions
- common project commands

Instead of installing packages globally and hoping everything works together, uv keeps each course repository in its own environment.

Why use uv

Students usually care about a few practical benefits:

- Reproducible environments: the same repository can be set up the same way on different computers.
- Easier setup: `uv sync` installs what the project needs without a long series of manual steps.
- Consistent package versions: students are less likely to have "it works on my machine" problems.
- Simpler command execution: `uv run ...` runs commands inside the correct project environment.
- Less confusion: it is easier than mixing global Python installs, random `pip install` commands, and multiple environments.

How uv fits into this course

In this course, uv is the default workflow.

Most of the time, you will:

1. Open a course repository.
2. Run `uv sync`.
3. Use `uv run ...` to run scripts, tests, and notebooks.

That means you should usually avoid installing course packages globally. If the course repository already has a `pyproject.toml` file, treat that repository as the source of truth for the environment.

Installing uv

Because students use different operating systems and package managers, there is not one single install command that fits everyone. Use one of the official installation methods below, then verify the install with `uv --version`.

Recommended ways to install

- Official standalone installer: good default if you want the method from the uv documentation.
- `pipx`: a good option if you already use Python tooling and want uv installed in an isolated tool environment.
- System package manager: a good option if you prefer managing software with your platform's package tools.

Official installer examples

If you are using a Unix-like shell:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If you are using PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/inst
```

Other common installation options

If you already use one of these tools, you may prefer:

```
pipx install uv
pip install uv
brew install uv
winget install --id=astral-sh.uv -e
scoop install main/uv
sudo port install uv
```

You do not need to use all of these. Pick the method that matches your system and the course setup guide.

Verify the installation

After installing uv:

1. Close and reopen your terminal if needed.
2. Run `uv --version`.
3. If the command is not recognized, check whether the install method added uv to your PATH.

Getting started

If uv is already installed, these are the first commands you will usually run in a course repository:

```
uv --version
uv sync
uv run python --version
uv run python src/setup_check.py
uv run jupyter lab
```

What these commands do:

- `uv --version` checks that uv is installed.
- `uv sync` creates or updates the project environment from the repository configuration.
- `uv run python --version` confirms which Python version the project is using.
- `uv run python src/setup_check.py` runs a setup verification script inside the course environment.
- `uv run jupyter lab` opens notebook work using the project environment.

If uv is not installed yet, use the installation section above or the official uv documentation before continuing.

Common commands

In this course, `uv sync` and `uv run` are the commands you will use most often. Commands like `uv add` and `uv init` are usually only used if an assignment or the instructor explicitly asks for them.

Command	What it does	Typical use in this course
<code>uv --version</code>	Shows the installed uv version	Quick check that uv is available

<code>uv sync</code>	Creates or updates the environment from project files	Standard first step after cloning a repo
<code>uv run ...</code>	Runs a command inside the project environment	Running scripts, tests, Python, and notebooks
<code>uv add package-name</code>	Adds a dependency to the project	Only when told to add a package
<code>uv remove package-name</code>	Removes a dependency from the project	Only when cleaning up a project dependency
<code>uv lock</code>	Updates the lockfile	Mostly instructor or project-maintainer work
<code>uv init</code>	Creates a new uv project	Only when starting a brand-new project

Examples:

```
uv run python script.py
uv run pytest
uv add pandas
uv remove pandas
uv lock
uv init
```

Typical course workflows

Syncing an existing class repository

1. Open a terminal in the repository folder.
2. Run `uv sync`.
3. Wait for the environment to finish setting up.
4. Run `uv run python --version` if you want to confirm the interpreter.

Running a Python script

1. Open a terminal in the repository folder.
2. Run `uv run python path/to/script.py`.
3. Read the output and fix any errors inside the project environment.

Example:

```
uv run python src/setup_check.py
```

Running tests

1. Open a terminal in the repository folder.
2. Run `uv run pytest`.
3. Read the failing test output carefully if something breaks.

Example:

```
uv run pytest
```

Opening notebook work

1. Open a terminal in the repository folder.
2. Run `uv run jupyter lab`.
3. Open the notebook you need.
4. In VS Code or Jupyter, make sure the notebook is using the environment created for the repository.

Example:

```
uv run jupyter lab
```

Common problems and fixes

uv is not installed or not on PATH

Symptoms:

- `uv` command not found
- terminal says `uv` is not recognized

What to try:

- Install `uv` using the course setup guide.
- Close and reopen the terminal after installation.
- Run `uv --version` again.

Wrong Python version

Symptoms:

- the course expects one Python version, but `uv run python --version` shows another
- package installation or import problems appear even after syncing

What to try:

- Check the expected Python version in the course instructions.
- Run `uv run python --version`.
- If it does not match the course expectation, ask for help before manually changing a lot of settings.

VS Code is using the wrong interpreter

Symptoms:

- code runs in the terminal but fails in VS Code
- imports work in one place and fail in another

What to try:

- Open the repository folder, not just an individual file.
- Use VS Code's interpreter selection tool and choose the interpreter for the course repository environment.
- Re-run the notebook or script after selecting the correct interpreter.

A package seems missing after `uv sync`

Symptoms:

- `ModuleNotFoundError`
- imports fail even though the repository should include the package

What to try:

- Make sure you are inside the correct repository folder.
- Run `uv sync` again.
- Run the command with `uv run ...` instead of plain python.
- Check whether the package is actually part of the repository configuration.

Notebook kernel mismatch

Symptoms:

- the notebook opens, but imports fail
- the notebook kernel name looks unfamiliar or does not match the course project

What to try:

- Reopen the notebook after running `uv run jupyter lab`.
- Select the kernel associated with the course repository environment.
- If needed, restart the kernel and run the notebook again from the top.

When to ask for help

Ask for help early if you are stuck for more than a short time. When you ask, include:

- operating system
- Python version
- uv version
- the exact command you ran
- the full error message as text
- what you already tried

That information makes it much easier for someone to help you quickly.